

数据库期末复习

巫宇轩 1911489

目录

数据库期末复习.....	1
考点和题型.....	5
.....	5
第一章.....	5
第二章 数据存放（不考核）	7
第三章 索引（不考核）	8
常规索引（稀疏索引 稠密索引 辅助索引）	8
删除插入操作.....	8
指针桶.....	8
B+树索引.....	8
注意点：	8
第四章 ER 模型（重点）	9
关系（1 对 1 等）	9
弱实体集合.....	9
ER 到关系模型.....	9
子类实体转换.....	10
数据模型.....	10
注意点：	10
第五章 SQL 语句.....	11
表的创建（重点）	11
插入 删除 更新.....	13
断言 触发器 存储.....	13
视图和索引.....	13
授权.....	14
注意点：	15
第六章 数据库模式设计（重点）	15
三种异常和冗余.....	15
函数依赖.....	16
key superkey.....	16
寻找 key.....	16
闭包.....	17
Armstrong 规则.....	18
表的拆分.....	18
范式：	19
方法 1：类似于 BCNF 的分解 没有达到进行分解.....	21
方法 2：	21
注意点：	24
第七章 关系代数（考）	25
并交差 笛卡尔积.....	25
选择语句.....	25
投影语句：	26
连接语句：先做笛卡尔积 再根据条件保留 tuple.....	26
自然连接：连接的两个表要有相同的属性.....	26

两个连接的区别：	27
除运算：	27
DELTA 用于去除包中重复元组.....	28
TAU 排序.....	29
广义投影.....	29
GAMMA 分组和聚合.....	29
外连接.....	30
重命名：	30
运算优先级：	31
表达式树：	31
注意点：	32
第八章 查询处理（细节不考）	32
逻辑查询计划：	33
物理查询计划：	34
关系代数变化规则.....	34
物理查询计划代价估计.....	36
连接的处理方案.....	36
第九章 故障恢复.....	39
事物.....	39
undo 日志.....	40
redo 日志：	43
undo/redo 日志：	45
非静止的检查点：	46
日志的备份：	48
第十章 并发控制.....	48
调度：	49
冲突等价调度：	50
可串行化调度：	50
重要结论：	50
求等价的串行化调度：	50
两阶段锁：	51
共享/排他锁：	52
新的规则：	52
更新锁：	52
警示协议：	54
时间戳（不考）：	55
有效性验证（不考）：	55
考题样例：	56
题目 1： ER.....	56
题目 2： sql.....	57
题目 3： 范式.....	57
题目 4： 数据故障恢复.....	58
题目 5： 并发控制.....	59
题目 6： 求函数依赖和 key.....	61

考点和题型

- Introduction to DBMSs (重点知晓数据库系统的三层模式结构和两层映像的功能)
- Principles of Data Layout and Index (本章不考核, SimpleDB实现涉及该部分内容)
- The Entity Relationship model
- E/R to Relational Transformation
- SQL: Introduction
- SQL: Advanced
- DB Schema Design (考核所有知识点)
- Relation Algebra & Query Processing (关系代数基本操作, 查询处理细节不考核)
- Crash Recovery (考核所有知识点)
- Concurrency Control (掌握优先图、两段锁等并发思想, Timestamp & Validation 不考核)



期末考试重点

- Data Models (ER转关系, SQL DDL, 主外键约束)
- Join query semantics (写SQL查询结果)
- Normal Form (候选码, 3NF, BCNF, FD, 无损连接)
- Relational Algebra and SQL Queries (关系代数、SQL查询)
- Concurrency Control (优先图, 冲突可串行化调度, 两段锁)
- Transaction Management (undo/redo logging)
- 简答题 (数据库架构, SQL授权, 意向锁, 多值依赖, 逻辑优化等)

逻辑优化 (IO 代价) sql->关系代数->代价估计

第一章

DBMS 三层模式架构 (模式层)

Physical level (物理模式、内模式、存储模式)

how are these tables stored, how many bytes/attribute etc, indices, hash, ...

Logical level (模式、逻辑模式)

describes data stored in database, and the relationships among the data.

STUDENT(学号, 姓名, 出生日期, 所属学院)

TAKES (学号, 课号, 成绩)

View level (外模式、子模式、用户模式)

application programs hide details of data types. Views can also hide information (e.g., salary) for security purposes.

两层映像实现两层独立性

- Logical data independence (逻辑独立性)
 - protection from changes in the logical structure of the data
 - can add (drop) column; add/drop table
- Physical data independence (物理独立性)
 - protection from physical layout changes
 - can add index; change record order

DML:

查询和更新指令构成了 SQL 的 DML 部分:

- * SELECT - 从数据库表中获取数据
- * UPDATE - 更新数据库表中的数据
- * DELETE - 从数据库表中删除数据
- * INSERT INTO - 向数据库表中插入数据

DDL:

SQL 中最重要的 DDL 语句:

- * CREATE DATABASE - 创建新数据库
- * ALTER DATABASE - 修改数据库
- * CREATE TABLE - 创建新表
- * ALTER TABLE - 变更 (改变) 数据库表
- * DROP TABLE - 删除表
- * CREATE INDEX - 创建索引 (搜索键)
- * DROP INDEX - 删除索引

第二章 数据存放 (不考核)

稀疏索引更快

稀疏索引针对的数据需要有序 无序的必须使用稠密索引

record 即为 tuple

scheme 为关系模式既包括表名和每列的名称

第三章 索引（不考核）

常规索引（稀疏索引 稠密索引 辅助索引）

删除插入操作

指针桶

B+树索引

Relation R has 4000 records. Suppose a B+ tree index is built on R' s key attribute. A block can hold a B+ tree node (interior or leaf node) with 20 keys and 21 pointers. In this case, at least [填空1] blocks are needed to hold the entire B+ tree index?

记录放到了同一层 200 块存放 4000 条记录 10 块指向这 200 块，1 块指向这 10 块
最终 $200+10+1=211$

注意点：

注意主索引直接指向数据块

任何索引的 key 都需要排序 这里指索引中的 key

b+树和 b 树不同之处在于叶子的最小指针是向下取整的

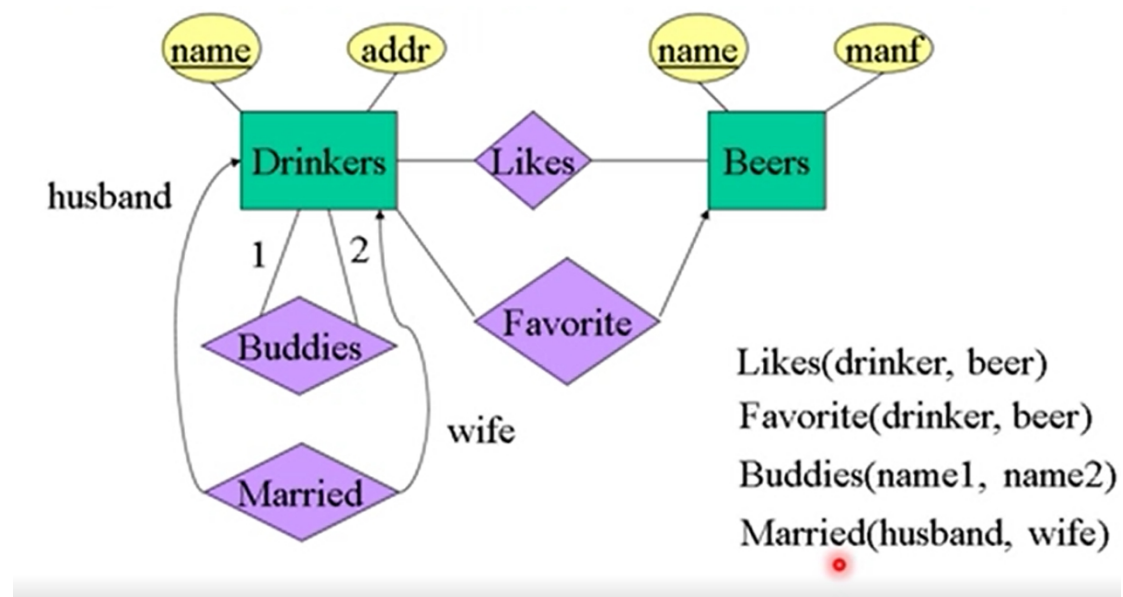
第四章 ER 模型（重点）

关系（1 对 1 等）

弱实体集合

ER 到关系模型

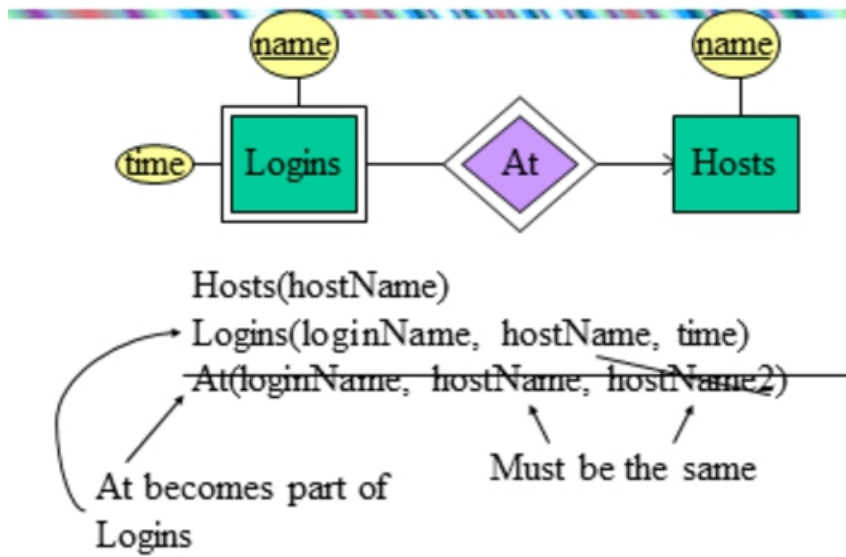
实体的转换 联系的转换



表的合并（多对 1 关系表和多的表合并）

弱实体转换时，需要包含所有属性和依赖的主 key

Another Example



子类实体转换

OO ER NULL 三种

OO 方法：最节省空间 每个实体只会出现一次 不同实体集合在一张表中

ER 方法 name 需要多次存储 现在父表存储 再在子表存储 查询效率最低

数据模型

实体完整性：主键不重复不为 null

参照完整性：外键要么为 null 要么是其他关系中出现的

用户定义完整性：满足用户的约束

注意点：

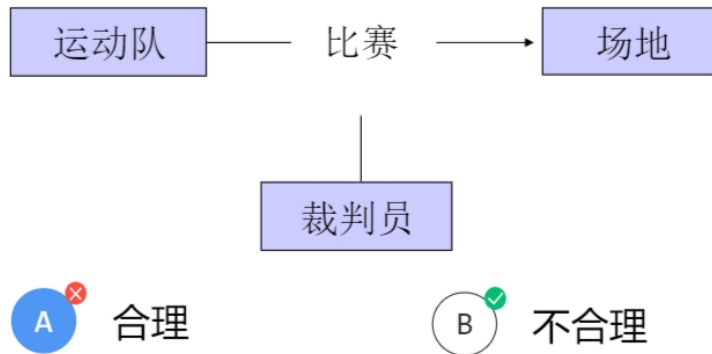
1 对 1 多对多的箭头，有箭头的对立方即为 1

尖箭头表示至多 1 个 即对应为 1 或者 0 个



圆箭头：出发的实体集合中每个实体都对应另一个集合中一个实体

关系是关联的实体集合的笛卡尔积的子集



运动队的比赛可能多场地举行

子实体的 key 有 root 的 key1 决定

弱实体集合的支持联系只能是多对 1 的联系

第五章 SQL 语句

表的创建（重点）

可选类型

char(n)/character(n)
 varchar(n)/character varying
 int/integer
 smallint
 numeric(p,d)
 real,double precision

修改已有表的某一行

Examples

```
ALTER TABLE Bars ADD phone CHAR(16)
DEFAULT 'unlisted';
```

```
ALTER TABLE Bars DROP COLUMN license;
```

多个主键需要合并在一起

• The bar and beer together are the key for Sells:

```
CREATE TABLE Sells (
    bar      CHAR(20),
    beer     VARCHAR(20),
    price    REAL,
    PRIMARY KEY (bar, beer)
);
```

• Example:

```
CREATE TABLE Beers (
    name CHAR(20) UNIQUE,
    manf CHAR(20)
);
```

外键单个使用 reference

```
CREATE TABLE Beers (
    name  CHAR(20) PRIMARY KEY,
    manf  CHAR(20) );
CREATE TABLE Sells (
    bar    CHAR(20),
    beer   CHAR(20) REFERENCES Beers(name),
    price  REAL );
```

```
CREATE TABLE Beers (
    name   CHAR(20) PRIMARY KEY,
    manf   CHAR(20) );
```

```
CREATE TABLE Sells (
    bar     CHAR(20),
    beer    CHAR(20),
    price   REAL,
    FOREIGN KEY (beer) REFERENCES
    Beers(name) );
```

多个外键:

```

Create table Frequents(
    drinker char(10),
    bar char(20) ,
    foreign key(drinker) references Drinkers(name),
    foreign key(bar) references Bars(name),
    primary key(drinker,bar));

```

插入 删除 更新

断言 触发器 存储

请写出创建触发器的语句，使得如下SQL语句可以如下所示表上正确执行

UPDATE workson SET empno=11111 WHERE empno=9031;

Employee

empno	empname	deptno
2581	徐唱	d2
9031	李静	d2
10102	王闻刚	d3
18316	冯新	d1
25348	张风	d3
28559	刘国凤	d1
29346	赵东生	d2

Workson

empno	projectno	job	enterdate
2581	p3	分析员	98-10-15
9031	p1	管理员	98-4-15
9031	p3	职员	97-11-15
10102	p1	分析员	97-1-10
10102	p3	管理员	99-1-1
18316	p2	职员	98-2-15
25348	p2	<NULL>	98-6-1
28559	p1	<NULL>	98-8-1
28559	p2	职员	99-2-1
29346	p1	职员	98-1-4
29346	p2	<NULL>	97-12-15

```

delimiter //
create trigger WorksonUpdateTrig
BEFORE UPDATE ON workson FOR EACH ROW
BEGIN
    IF NEW.empno NOT IN (SELECT empno From employee) THEN
        INSERT INTO employee(empno) values (new.empno);
    END IF;
END; //
delimiter ;

```

能否直接将employee中的9031直接更新为11111? 不能，因为在workson更新前不能删除9031，否则违反外键约束，而触发器不能设定为workson更新后，因为workson无法更新。可考虑增加after update on workson的触发器，如果old.empno在employee表中，则将employee表中empno为old.empno的元组的empname和deptno的值更新至employee表中empno为新.empno的元组，再将employee表中empno为old.empno的元组删除。

视图和索引

视图是虚拟的关系 并不是以图的形式存在

视图创建需要用 `select` 列名要么不指定要么全部指定

授权

可以授权的 `select update insert delete` 注意没有 `from`

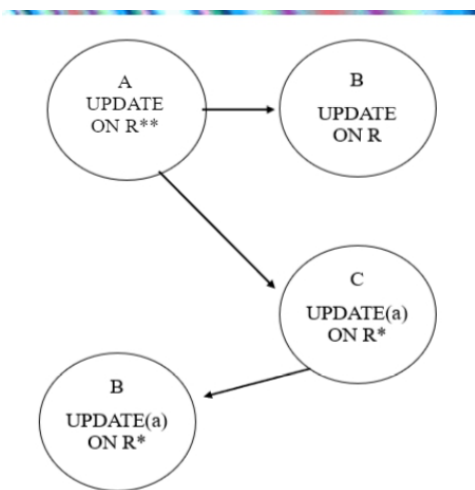
```
GRANT SELECT, UPDATE(price)
ON Sells
TO sally;
```

with `grant option` 可以让其继续授权他人
收回特权 注意收回时分为级联和拒绝

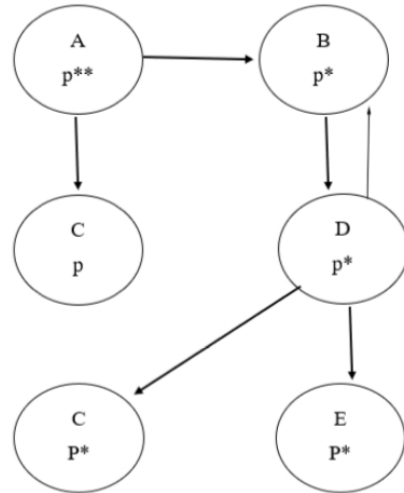
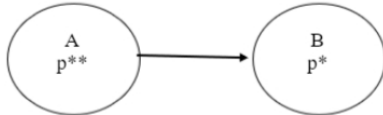
```
REVOKE <list of privileges>
ON <relation or other object>
FROM <list of authorization ID' s>;
```

授权图（重点）

**代表 owner *代表可以授权他人 其他的代表有权利



- 1 A GRANT p TO B WITH GRANT OPTION
- 2 A GRANT p TO C
- 3 B GRANT p TO D WITH GRANT OPTION
- 4 D GRANT p TO B, C, E WITH GRANT OPTION
- 5 B REVOKE p FROM D CASCADE
- 6 A REVOKE p FROM C CASCADE



注意点：

对于关系Movies(title,year,length,genre,studioName),查询select * from Movies where length <= 120 or length > 120; 会返回Movies表中所有元组，这种说法

(A) 正确

(B) 错误

(C) 不一定

非 key 属性可以为 null 无法比较大小

存储历程可以是一个存储函数或者是存储过程

unique 可以出现 null 但是只能用一个

可以用多个 unique 但是 primary 只能有一个
外键 reference

主键有多个的时候，需要统一括号内部声明

第六章 数据库模式设计（重点）

三种异常和冗余

数据冗余(想修改一个属性,就要更新多行数据)

插入异常(想要插入数据,结构因为表设计的问题,导致不能成功插入)

删除异常(只想删除其中的某些数据 ,结果把不该删的也删了)

更新异常(想更新一条数据,结果工作量大,还容易出错)

根据南开大学的教学教务应用场景,你觉得关系模式

学生(学号、课号、课程名称、成绩)存在哪些异常(Anomalie)?



插入异常



删除异常



更新异常

插入:想录入一个课程,但是必须添加一个完整的学生

删除:删除了学生,可能有些课程都被删了,就不在数据库了

更新:改课程名称,需要将所有地方都修改

函数依赖

函数依赖要求满足函数性质 即一对一 多对一 但不可以一对多

key superkey

key 可以函数决定关系表中所有属性

key 的所有子集都无法决定所有属性

superkey (包含 key) 可以决定所有属性

关系模式 $R(ABC), F=\{A \rightarrow B, B \rightarrow A, A \rightarrow C\}$, R最高属于第几范式?

superkey 为 $\{A B AB ABC\}$

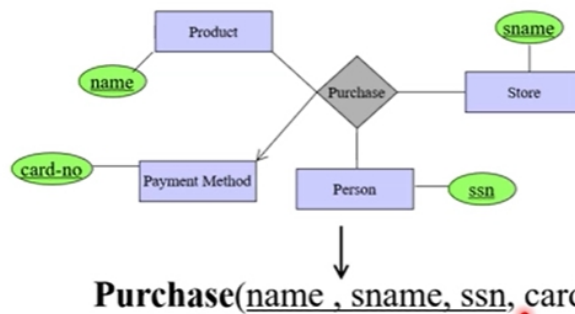
key 为 $\{A B\}$

寻找 key

实体的 key 或者多对多关系中关联实体的 key

如果是多对 1 关系 则 key 不需要包含 1 端的

But: if there is an arrow from the relationship to E, then we don't need the key of E as part of the relation key.



求 key 的算法

求键的启发式算法：①不在任何函数依赖右边出现的属性必是键的一部分；
②不在函数依赖左部出现的属性不是键的一部分。
(一定要先使用① 后再使用②)

$R(A,B,C,D,E,F)$

FDs: $\{AB \rightarrow C, AD \rightarrow E, B \rightarrow D, AF \rightarrow B\}$

① A、F属性没有在函数依赖的右边出现，它必是键的一部分，先计算 $(A,F)^+$ ，如果它不是key，再继续计算 $(A,F,...)^+$

$R(A,B,C,D,E)$

FDs: $\{AB \rightarrow CE, E \rightarrow AB, C \rightarrow D\}$

①所有属性都在函数依赖的右边出现了，②左边只出现了A、B、C和E，则不需要计算 $(D,...)^+$ ，D肯定不在keys中出现

闭包

求函数依赖集合的闭包：

S 是一个函数依赖集合

循环：

利用自反和增广添加新的依赖

对每一对依赖 使用传递 添加新依赖

until:

无法添加任何依赖

闭包要包含逻辑蕴含的函数依赖

求属性闭包的算法：

将已有的 n 个属性放入 x 中 即属性闭包一定要包含自身

循环：

计算寻找新的属性加入 x 中

until 无法寻找到新属性

Armstrong 规则



Armstrong's Axioms

- Reflexivity rule
 - $A_1A_2\dots A_n \rightarrow$ a subset of $A_1A_2\dots A_n$
- Augmentation rule
 - $A_1A_2\dots A_n \rightarrow B_1B_2\dots B_m$, then
 $A_1A_2\dots A_n C_1C_2\dots C_k \rightarrow B_1B_2\dots B_m C_1C_2\dots C_k$
- Transitivity rule
 - $A_1A_2\dots A_n \rightarrow B_1B_2\dots B_m$ and
 $B_1B_2\dots B_m \rightarrow C_1C_2\dots C_k$, then
 $A_1A_2\dots A_n \rightarrow C_1C_2\dots C_k$

1 自反性: 集合 s 可以决定子集, 右边的可以是左边的子集

2 增广律: 可以加上一些属性

3 传递律

推导出的规则:

- Union rule
 - $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
 - (X, Y, Z are sets of attributes)
- Decomposition rule
 - $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$
- Pseudo-transitivity rule
 - $X \rightarrow Y$ and $YZ \rightarrow U$, then $XZ \rightarrow U$

结合律 分解律 伪传递律

表的拆分

两个表之间连接的属性应该为 key 否则会出现冗余

拆分后要考虑逻辑蕴含的函数依赖

例如:

$R(A\ B\ C\ D)$ B-C C-D

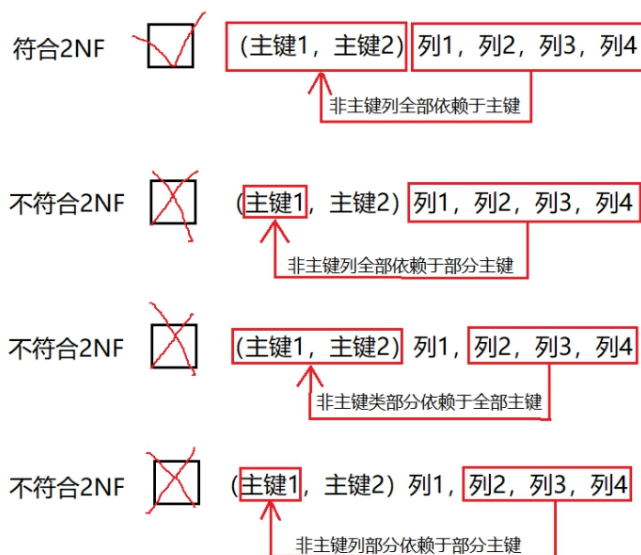
$R_1(D\ B)$ 包含 B-D

范式：

达不到范式要求的后果：

1NF： 表中每个属性不可分（创造 **create** 后即达到）

2NF： 不存在非主属性对于码的部分函数依赖



BCNF：

即只要有函数依赖 该依赖为非平凡 且左边为 superkey
如果出现不满足的依赖 即不为 BCNF

非平凡函数依赖：A-》B 则 B 不可以为 A 子集

不存在任何依赖->BC 范式

只有 2 个属性的表必定是 BC 范式

构造 BCNF：

1 选择不满足的依赖的左边关系 X 以 X 的闭包作为新的表 另一张表中也要包含 X

Drinkers(name, addr, beersLiked, manf, favBeer)

$F = \text{name} \rightarrow \text{addr}, \text{name} \rightarrow \text{favBeer}, \text{beersLiked} \rightarrow \text{manf}$

- Pick BCNF violation $\text{name} \rightarrow \text{addr}$.
- Close the left side: $\{\text{name}\}^+ = \{\text{name}, \text{addr}, \text{favBeer}\}$.
- Decomposed relations:
 1. Drinkers1(name, addr, favBeer)
 2. Drinkers2(name, beersLiked, manf)

2 再判断分解后的表是否为 BC 范式

- For Drinkers1(name, addr, favBeer), relevant FD's are $\text{name} \rightarrow \text{addr}$ and $\text{name} \rightarrow \text{favBeer}$.
 - Thus, {name} is the only key and Drinkers1 is in BCNF.

发现第二个表不满足 再分解

- For Drinkers2(name, beersLiked, manf), the only FD is $\text{beersLiked} \rightarrow \text{manf}$, and the only key is {name, beersLiked}.
 - Violation of BCNF.
- $\text{beersLiked}^+ = \{\text{beersLiked}, \text{manf}\}$, so we decompose Drinkers2 into:
 1. Drinkers3(beersLiked, manf)
 2. Drinkers4(name, beersLiked)

注意 **bcnf** 无法保证全部的函数依赖 所以需要检查 如果不满足全部函数依赖 则无法分解成 **bcnf**

BC 范式作用:

1 去掉部分冗余

2 保证无损连接

但是可能会丢去函数依赖

3NF:

如果未达到 **bcnf** 则可以判断是否达到 **3nf**

主属性：包含在关系模式 key 属性中的属性

违反 3nf 情况：

$x \rightarrow y$ x 不是 superkey 且 y 不是主属性（可见前半部分是 bcnf 只要一个部分满足即可）

重要结论：

- 1 任何一个关系模式在可恢复性前提下 分解成多个 bcnf 集合
- 2 任何一个关系模式可以在保持恢复性、依赖关系条件下 分解成多个 3nf
- 3 但是无法在恢复性、依赖性条件下 分解成 bcnf 集合

3NF 分解（重点）：

方法 1：类似于 BCNF 的分解 没有达到进行分解

方法 2：

概念：最小函数依赖集 右边一个属性 且 其他依赖无法推出此依赖



最小函数依赖集

如果函数依赖集合 F 满足如下条件，则称 F 为一个极小函数依赖集，也称为最小依赖及或最小覆盖。

- (1) F 中的任意函数依赖的右部仅含有一个属性；
- (2) F 中不存在这样的函数依赖 $X \rightarrow A$ ，使得 F 与 $F - \{X \rightarrow A\}$ 等价；
- (3) F 中不存在这样的函数依赖 $X \rightarrow A$ ， X 有真子集 Z ，使得 $F - \{X \rightarrow A\} \cup \{Z \rightarrow A\}$ 与 F 等价。

如果 $G^+ = F^+$ ，就称 F 与 G 等价。 $G^+ = F^+$ 的充分必要条件是 $F \subseteq G^+$ 和 $G \subseteq F^+$

2（即该函数依赖可以被其他依赖推导 则该函数要省去）

3（某个依赖左边如果部分属性可以推导出右边 则该依赖需要省去）

已知关系模式和函数依赖集合，保持 FD 和无损连接 3NF 模式分解的算法如下：

- ① 求极小函数依赖集 $S_{\min}$ 和关系模式的所有键（keys）；
- ② 在 $S_{\min}$ 中按函数依赖左部相同原则进行分组，每个组中的所有属性形成分解后的子关系模式， $R_1, R_2, \dots, R_m$
- ③ 如果某个关系模式 R_i 的所有属性被另一个关系模式 R_j 所包含，删除关系模式 R_i ；
- ④ 判断是否有某个 key 出现在其中的一个关系模式中，如果出现则结束；如果没有出现，将任意一个 key 也作为子关系模式 R_k 加入到分解后的模式中。

举例：

- $S(SNO, SNAME, SEDPT, MN, CNAME, G)$
- $F = \{SNO \rightarrow SDEPT, SNO \rightarrow SNAME, SDEPT \rightarrow MN, SNO \rightarrow MN, (SNO, SNAME) \rightarrow SDEPT, (SNO, CNAME) \rightarrow G\}$

① 求最小函数依赖集合和所有键

$F_{min} = \{SNO \rightarrow SDEPT, SNO \rightarrow SNAME, SDEPT \rightarrow MN, (SNO, CNAME) \rightarrow G\}$
键key为 $(SNO, CNAME)$

② 函数依赖左部相同分组

$\{SNO \rightarrow SDEPT, SNO \rightarrow SNAME\}$ $S1(SNO, SDEPT, SNAME)$

$\{SDEPT \rightarrow MN\}$ $S2(SDEPT, MN)$

$\{(SNO, CNAME) \rightarrow G\}$ $S3(SNO, CNAME, G)$

③ 不存在子模式的包含情况； ④ $S3$ 中含有key，分解终止。

无损分解判断：



判断无损连接分解的算法

- 已知 $R \langle U, F \rangle$, $U = \{A, B, C, D, E\}$
- $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow E\}$
- R 的一个分解 $R1(A, B, C), R2(C, D), R3(D, E)$

	A	B	C	D	E
$R1(A, B, C)$	a_1	a_2	(a_3)	b_{41}	a_5
$R2(C, D)$	b_{21}	b_{22}	(a_3)	(a_4)	b_{25}
$R3(D, E)$	b_{31}	b_{32}	b_{33}	a_4	a_5

1 将关系对应属性设计成矩阵。出现用 a 表示 未出现用 b 表示

2 扫描矩阵和依赖关系

根据函数依赖 $AB \rightarrow C$ 则 AB 相等则 C 必须相等

同理 $C \rightarrow D$ 同样的 a_3 对应的 D 要相同才行 将第一行的 b_{41} 改成 a_4

3 如果某一行都是 a 则为无损连接分解，如果没有，则继续扫描直到数据不变化为止，此时仍然没有一行都是 a 则不为无损连接。

原理：如果分解后的依赖关系无论怎么拼接都无法推出所有属性，则有损失

多值依赖：关系表中存在一个以上多对多联系

name	phones	beers
张三	{p1}	{b1}
	{p2}	{b2}
李四	{p1}	{b2}
	{p3}	{b4}
		{b5}

人和电话 人和啤酒之间都为多对多 而电话和啤酒独立

定义：

$x \twoheadrightarrow y$ 则对应的 2 个 x 相同的 tuple 中交换 y 的值 结果已经存在于表中

BCNF 减少部分冗余 但是无法减少多值依赖的冗余

注意多值依赖不属于函数依赖 所以不需要用于判断 **bcnf**，但是函数依赖属于多值依赖

且如果 $x \twoheadrightarrow y$ x 和 y 都为 key

如果 $x \twoheadrightarrow y$ 则该 tuple 的其他属性也多值依赖于 x

多值依赖不可拆分：

If $AB \twoheadrightarrow C$ then $A \twoheadrightarrow C$, $B \twoheadrightarrow C$ 错误

If $A \twoheadrightarrow BC$ then $A \twoheadrightarrow B$, $A \twoheadrightarrow C$ 正确

If $AB \rightarrow \rightarrow C$ then $A \rightarrow \rightarrow C$, $B \rightarrow \rightarrow C$ 错误

If $A \rightarrow \rightarrow BC$ then $A \rightarrow \rightarrow B$, $A \rightarrow \rightarrow C$ 错误

4NF:

非平凡的多值依赖：

Y 不是 X 的子集 X 和 Y 没有包含所有属性

4nf 定义 $x \twoheadrightarrow y$

所有非平凡多值依赖都满足 X 为 superkey（平凡依赖不用考虑）

如何寻找 key? ?

函数依赖同上 多值依赖两边都是 key

拆分 **4NF**：

和 **bcnf** 类似先找到违背 **4NF** 的多值依赖：

Drinkers(name, addr, phones, beersLiked)

FD: $\text{name} \rightarrow \text{addr}$

MVD's: $\text{name} \twoheadrightarrow \text{phones}$

$\text{name} \twoheadrightarrow \text{beersLiked}$

- Key is {name, phones, beersLiked}.
- All dependencies violate 4NF.

将违背多值依赖的放到一张表中

- Decompose using $\text{name} \rightarrow \text{addr}$:

1. Drinkers1(name, addr)

- In 4NF; only dependency is $\text{name} \rightarrow \text{addr}$.

2. Drinkers2(name, phones, beersLiked)

- Not in 4NF. MVD's $\text{name} \twoheadrightarrow \text{phones}$ and $\text{name} \twoheadrightarrow \text{beersLiked}$ apply. No FD's, so all three attributes form the key.

表二 key 为三个属性 左侧不为 superkey 需要继续分解

- Either MVD $\text{name} \twoheadrightarrow \text{phones}$ or $\text{name} \twoheadrightarrow \text{beersLiked}$ tells us to decompose to:
 - Drinkers3(name, phones)
 - Drinkers4(name, beersLiked)

注意点:

判断是否保存函数依赖，要看拆分后函数依赖的闭包是否和原来的相同

(1) 设关系模式R (A, B, C) 上有函数依赖 $F=\{AB \rightarrow C, A \rightarrow BC\}$, 则F的最小函数依赖集 $F_{min}=\{\text{_____}\}$, R的候选码是 _____。

(2) 设R(A, B, C, D), $F=\{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$, R的候选码有: _____。对分解 $r = \{R_1(A, B), R_2(B, C), R_3(C, D)\}$, 它_____ (是/不是) 无损连接, _____ (保持/不保持) 函数依赖。

拆分后每个表的关系为 A-B B-A B-C C-B C-D D-C 闭包和之前相同

第七章 关系代数（考）

并交差 笛卡尔积

R(A B C)	S(A B C)	R×S(R.A R.B R.C S.A S.B S.C)
2 5 7	2 6 9	2 5 7 2 6 9
2 6 9	2 8 9	2 5 7 2 8 9
3 6 7	3 6 7	2 5 7 3 6 7
		2 6 9 2 6 9
		2 6 9 2 8 9
		2 6 9 3 6 7
		3 6 7 2 6 9
		3 6 7 2 8 9
		3 6 7 3 6 7

R ∪ S(A B C)	R - S(A B C)
2 5 7	2 5 7
2 6 9	
3 6 7	
2 8 9	

R ∩ S(A B C)
2 6 9
3 6 7

选择语句

Sells

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Bud	2.50
Sue's	Miller	3.00

$JoeMenu := \sigma_{bar = "Joe's"}(Sells)$

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75

投影语句：

Sells

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Bud	2.50
Sue's	Miller	3.00

$Prices := \pi_{beer, price}(Sells)$

beer	price
Bud	2.50
Miller	2.75
Miller	3.00

连接语句：先做笛卡尔积 再根据条件保留 tuple

<i>Sells</i> (	bar,	beer,	price)	<i>Bars</i> (	name,	addr)
	Joe's	Bud	2.50		Joe's	Maple St.
	Joe's	Miller	2.75		Sue's	River Rd.
	Sue's	Bud	2.50			
	Sue's	Coors	3.00			

$BarInfo := Sells \bowtie_{Sells.bar = Bars.name} Bars$

bar,	beer,	price,	name,	addr
Joe's	Bud	2.50	Joe's	Maple St.
Joe's	Bud	2.50	Sue's	River Rd.
Joe's	Miller	2.75	Joe's	Maple St.
Joe's	Miller	2.75	Sue's	River Rd.
Sue's	Bud	2.50	Joe's	Maple St.
Sue's	Bud	2.50	Sue's	River Rd.
Sue's	Coors	3.00	Joe's	Maple St.
Sue's	Coors	3.00	Sue's	River Rd.

自然连接：连接的两个表要有相同的属性

Sells(	bar,	beer,	price	)	Bars(	bar,	addr	)
	Joe's	Bud	2.50			Joe's	Maple St.	
	Joe's	Miller	2.75			Sue's	River Rd.	
	Sue's	Bud	2.50					
	Sue's	Coors	3.00					

BarInfo := Sells ⋈ Bars

Note Bars.name has become Bars.bar to make the natural join "work."

BarInfo(	bar,	beer,	price,	addr	)
	Joe's	Bud	2.50	Maple St.	
	Joe's	Milller	2.75	Maple St.	
	Sue's	Bud	2.50	River Rd.	
	Sue's	Coors	3.00	River Rd.	

两个连接的区别：

自然连接会把相同列合并 普通连接不合并

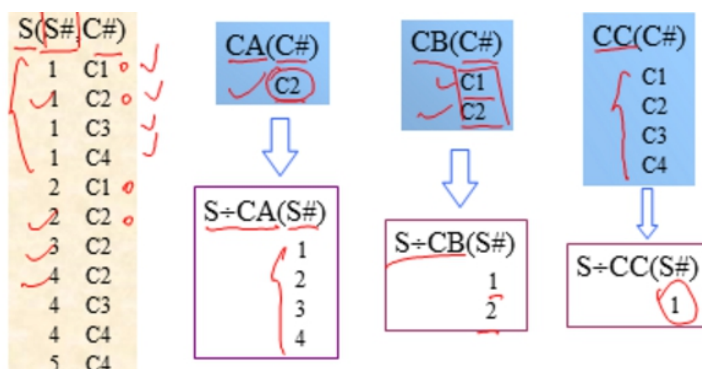
已知关系模式R(A,B)和S(B,C)，有以下两个关系代数运算：

Q1: $R \bowtie S$ Q2: $R \bowtie_{R.B=S.B} S$

请问下列哪个描述是正确的

- ☒ A Q1和Q2产生的结果一样
- ☐ B Q1的结果总是包含Q2的结果
- ☐ C Q2的结果总是包含Q1的结果
- ☒ D Q1和Q2产生不同的结果

除运算：



S(S#, SN, SD, SA)			
S1	A	CS	20
S2	B	CS	21
S3	C	MA	19
S4	D	CI	19
S5	E	MA	20
S6	F	CS	22

C(C#, CN, PC#)		
C1	G	—
C2	H	C1
C3	I	C1
C4	J	C2
C5	K	C4

SC(S#, C#, G)		
S1	C1	A
S1	C2	A
S1	C3	A
S1	C5	B
S2	C1	B
S2	C2	C
S2	C4	C
S3	C2	B
S3	C3	C
S3	C4	B
S4	C3	D
S4	C5	A
S5	C2	C
S5	C3	B

6. 查询选修了全部课程的学生号码和姓名

$$\pi_{s\#,sn}(S \bowtie (\pi_{s\#,c\#}(SC) \div \pi_{c\#}(C)))$$

对以下关系，查询不学 'C2' 课的学生姓名和年龄

S(S#, SN, SD, SA)			
S1	A	CS	20
S2	B	CS	21
S3	C	MA	19
S4	D	CI	19
S5	E	MA	20
S6	F	CS	22

C(C#, CN, PC#)		
C1	G	—
C2	H	C1
C3	I	C1
C4	J	C2
C5	K	C4

SC(S#, C#, G)		
S1	C1	A
S1	C2	A
S1	C3	A
S1	C5	B
S2	C1	B
S2	C2	C
S2	C4	C
S3	C2	B
S3	C3	C
S3	C4	B
S4	C3	D
S4	C5	A
S5	C2	C
S5	C3	B

先找到选了 C2 课的学生学号 用 delta 去重后 和学生表求差集
最后和 S 自然连接 然后投影到目标列

DELTA 用于去除包中重复元组

R =

A	B
1	2
3	4
1	2

$\delta(R) =$

A	B
1	2
3	4

TAU 排序

按照给定的属性列表进行排序 如果排列的属性相同 则任意排列
默认升序排列

R =

A	B
1	2
3	4
5	2

$$T_B(R) = [(5,2), (1,2), (3,4)]$$

广义投影

R =

A	B
1	2
3	4

$$\pi_{A+B, A, A}(R) =$$

A+B	A1	A2
3	1	1
7	3	3

GAMMA 分组和聚合

聚合不是关系代数

R =

A	B
1	3
3	4
3	2

SUM(A) = 7
COUNT(A) = 3
MAX(B) = 4
AVG(B) = 3

gamma 是先分组 再聚合

R =

A	B	C
1	2	3
4	5	6
1	2	5

Then, average *C* within groups:

$\gamma_{A,B,AVG(C)}(R) = ??$

First, group *R*:

A	B	C
1	2	3
1	2	5
4	5	6

A	B	AVG(C)
1	2	4
4	5	6

外连接

自然连接如果两边存在没有同时出现的属性，会丢失该 tuple，因此需要外连接

R =

A	B
1	2
4	5

S =

B	C
2	3
6	7

(1,2) joins with (2,3), but the other two tuples are dangling.

$R \bowtie S =$

A	B	C
1	2	3
4	5	NULL
NULL	6	7

重命名:

- The ρ operator gives a new schema to a relation.
- $R1 := \rho_{R1(A1,...,An)}(R2)$ makes *R1* be a relation with attributes *A1*,...,*An* and the same tuples as *R2*.
- Simplified notation: $R1(A1,...,An) := R2$.

Bars(

name	addr
Joe's	Maple St.
Sue's	River Rd.

)

$R(\text{bar}, \text{addr}) := \text{Bars}$

R(

bar	addr
Joe's	Maple St.
Sue's	River Rd.

)

运算优先级：

一元（选择 映射 重命名）

笛卡尔积 连接

交集

并集 差集

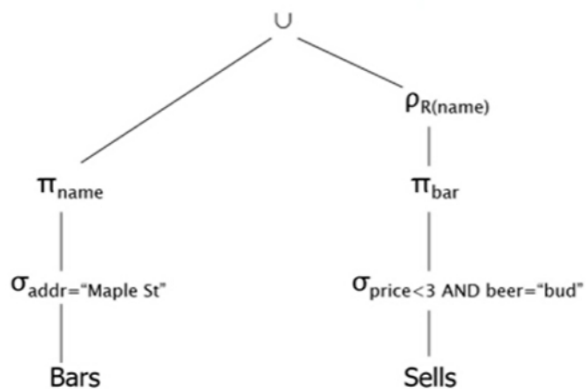
但是可以使用括号改变优先级

表达式树：

叶节点为操作数（关系） 内部节点为操作符

计算时从下向上

Using the relations Bars(name, addr) and Sells(bar, beer, price), find the names of all the bars that are either on Maple St. or sell Bud for less than \$3.



注意点：

并交叉的关系模式必须相同

注意笛卡尔积中出现同名属性 需要带上关系名进行区分

自然连接会合并同名称的属性

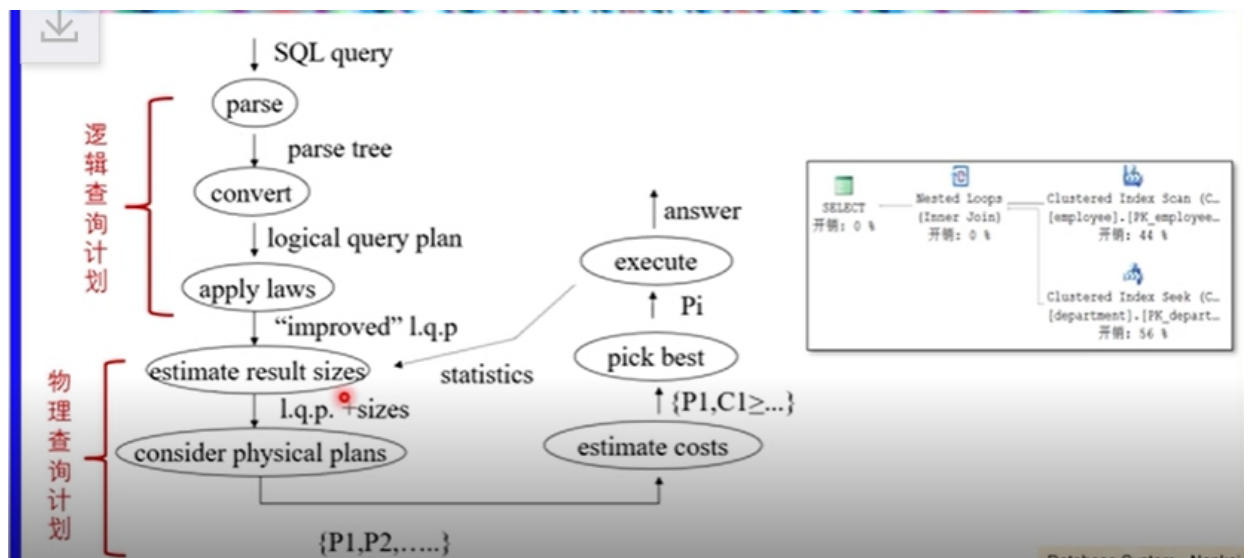
最小完备集：

并、差、笛卡尔积、选择、投影

$\cup$ 、 $-$ 、 $\times$ 、 σ 、 π

因为交可以由并 差代替

第八章 查询处理（细节不考）



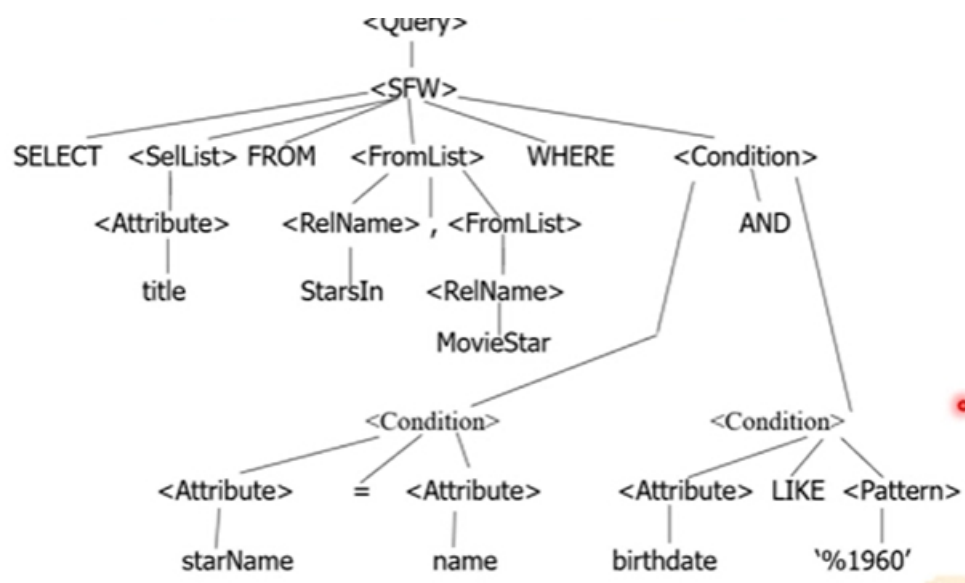
流程：

- 1 解析 sql 语句
- 2 生成逻辑查询计划
- 3 逻辑优化
- 4 估计中间运算结果大小

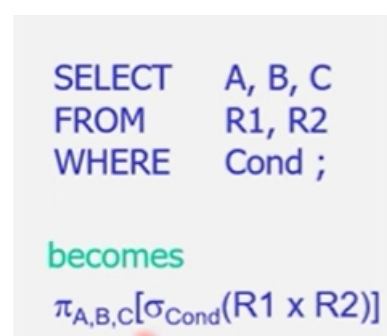
- 5 生成物理查询计划
- 6 执行最佳查询计划

逻辑查询计划：

1 sql 解析

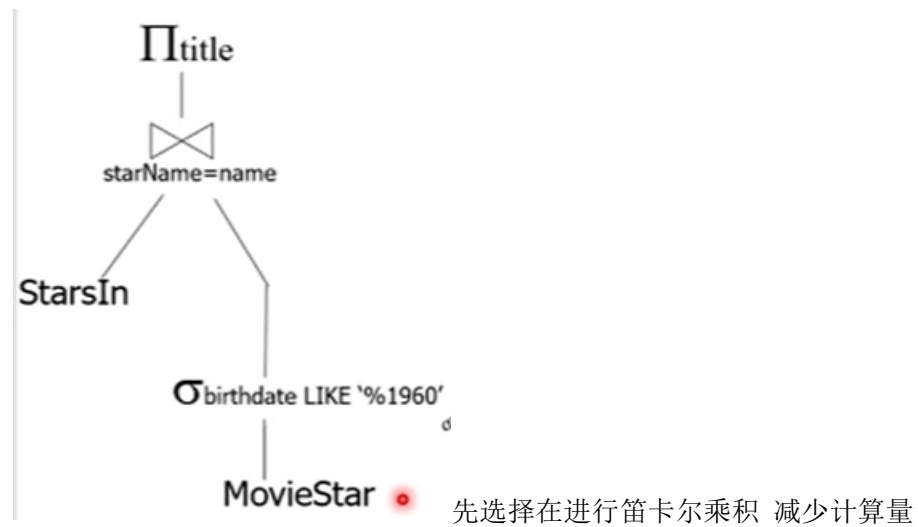


2 生成逻辑查询计划（关系代数表达式）



select 为投影 连接为笛卡尔积 where 为选择运算

3 等价变换规则转换成高效树



物理查询计划：

4 估计结果尺寸（估计树上各个节点大小）

5 生成查询计划

6 从查询计划中 找出最好算法并执行 并返回用户最终结果

关系代数变化规则

选择 投影运算一般放到叶子节点 越早做越好

natural join 笛卡尔积 满足交换结合

选择运算:

一般从左到右转换 即先求满足条件 1 再求满足条件 2

$$1. \sigma_{p1 \wedge p2}(R) = \sigma_{p1} [\sigma_{p2}(R)]$$

$$2. \sigma_{p1 \vee p2}(R) = [\sigma_{p1}(R)] \cup [\sigma_{p2}(R)]$$

笛卡尔积和 join 的转换:

从右到左

Definition of Natural Join:

$$R \bowtie S = \pi_L[\sigma_C(R \times S)];$$

Condition C equates attributes common to R and S, and π_L projects one copy of them out

Applied right-to-left

- definition of general join applied similarly

选择和自然连接结合:

Rules: σ + $\bowtie$ combined

Let p = predicate with only R attribs

q = predicate with only S attribs

m = predicate with attribs common to R, S

$$\sigma_p (R \bowtie S) = [\sigma_p (R)] \bowtie S$$

$$\sigma_q (R \bowtie S) = R \bowtie [\sigma_q (S)]$$

$$\sigma_{p \wedge q} (R \bowtie S) = [\sigma_p (R)] \bowtie [\sigma_q (S)]$$

$$\sigma_{p \wedge q \wedge m} (R \bowtie S) = \sigma_m \left[(\sigma_p R) \bowtie (\sigma_q S) \right]$$

$$\sigma_{p \vee q} (R \bowtie S) = \left[(\sigma_p R) \bowtie S \right] \cup \left[R \bowtie (\sigma_q S) \right]$$

物理查询计划代价估计

主要：磁盘 IO 代价

假设：1 忽略写盘操作 2 读取时一次读取一个数据块

连接的处理方案

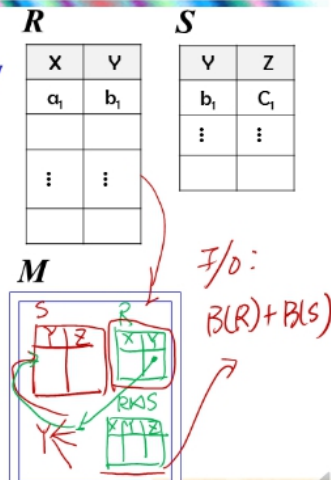
1 one pass join

将较小的表放入内存中（要求内存大于小表的大小）



One-Pass Join

- Requirement: $B(S) < M$, i.e., S fits in memory
- Read entire S in memory (using one buffer);
Build a dictionary (balanced tree, hash table) using join attributes of tuples as search key
- Read each block of R (using one buffer);
For each tuple t , find matching tuples from the dictionary, and output their join
- I/O cost: $B(S) + B(R) \leq$



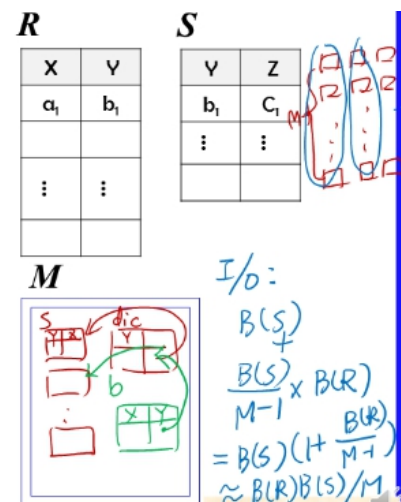
2 nested loop join

至少 2 个块的内存即可 针对内存不足

- If R and S clustered, can apply block-based nested-loop join:

```

for each chunk of  $M-1$  blocks of  $S$  do
  Read blocks in memory;
  Insert tuples in a dictionary using the join
  attributes;
  for each block  $b$  of  $R$  do
    Read  $b$  in memory;
    for each tuple  $r$  in  $b$  do
      Find matching tuples from the dictionary;
      output their join with  $r$ ;
  
```



将读到内存中 S 块上的连接属性建立索引

读取 R 中每一块 每一块中的 tuple 和内存中的 S 块进行匹配

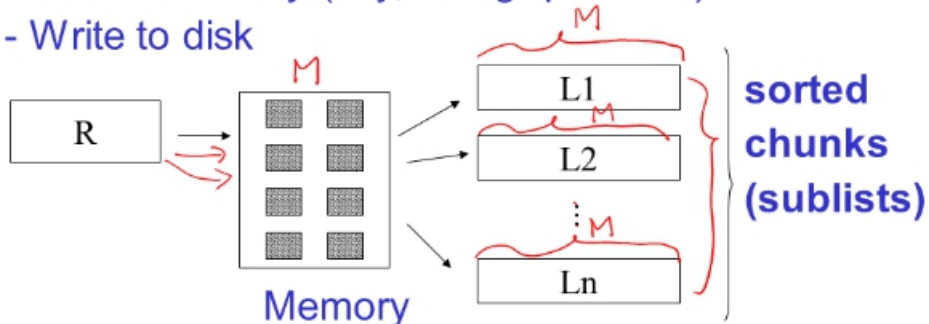
3 sort-based two-pass

先对连接属性排序再连接

排序过程:

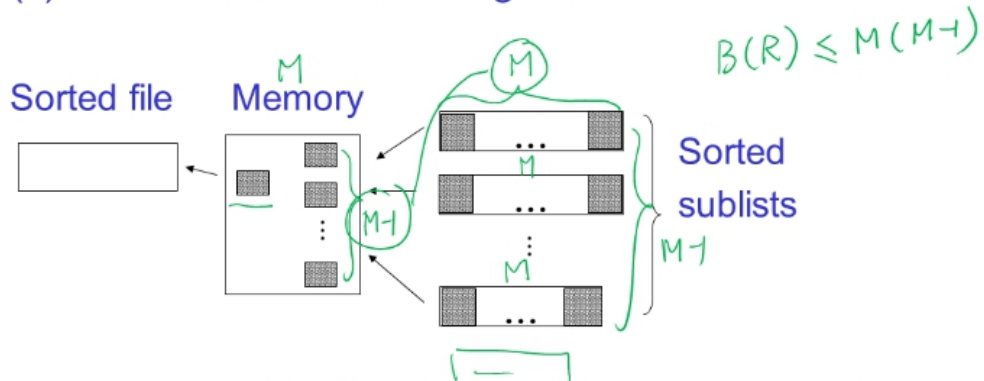
(i) For each main-memory-sized chunk of R:

- Read chunk
- Sort in memory (say, using quicksort)
- Write to disk



如果 R 的块数目大于主存 M，则需要将 R 分成多个序列，生成 L1 到 Ln 有序的子序列

(ii) Read all chunks + merge + write out



Always move the smallest head element to the output buffer

每次将 L1 到 Ln 中的第一块读入主存 选择最小的 输出到 sorted file，需要一块进行传输，因此 n 要小于等于 M-1

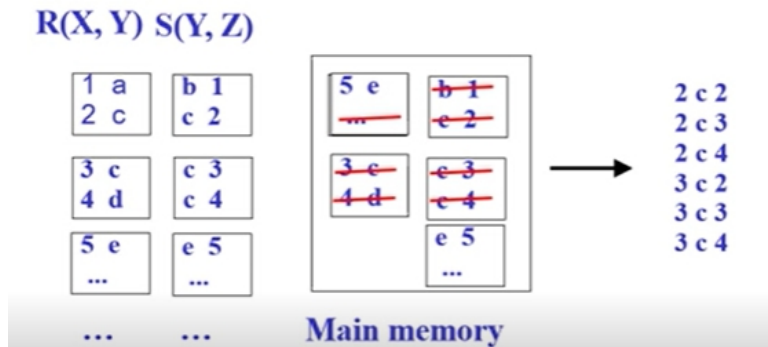
大小限制： $(M-1)*M$

排序后直接 join 即可：

1. Sort R with join attributes Y as the sort key;
2. Do the same for relation S;
3. Merge the sorted relations, using 1 buffer for current input block of each relation:

- skip tuples whose Y-value y not in both R and S
- read blocks of both R and S for all tuples whose Y value is y
- output all possible joins of the matching tuples
 $r \in R$ and $s \in S$

eg:



对 Y 排序后 读取 S 的第一块到内存中， b 不在 R 中跳过此 tuple
 c 在 R 中，输出匹配的 tuple，.....

IO 代价 $5(B(R)+B(S))$ ()

对于 R ：读入 M 一次 生成子表一次 排序时读入一次 输出一次
 连接时读入一次 一共 5 次

4 哈希的方法：

第九章 故障恢复

事物

由一系列读写组成 以原子单位提交或者终止

默认一个语句一个事物 多个事物要以 start transaction 开始 以 commit 或者 roolback 结束

A **transaction** is the DBMS's abstract view of a user program

- A sequence of **reads** and **writes** of database objects.
- Unit of work that must **commit** or **abort** as an **atomic unit**

• In "ad-hoc" SQL:
 - Default: each statement = one transaction

• In a program:
START TRANSACTION
 [SQL statements]
COMMIT or **ROLLBACK (=ABORT)**

```
DELETE FROM workson
WHERE empno = '10102'
SELECT * FROM workson
```

```
START TRANSACTION
DELETE FROM workson
WHERE empno = '10102'
SELECT * FROM workson
COMMIT [ROLLBACK]
```

特性:

A 原子性: 要么全做要么不做

C 一致性: 数据一致性

I 隔离性: 一个事物的操作不能被并发事物干扰

D 持久性: 一旦提交改变永久性

故障恢复 原子性 持久性

并发控制机制 一致性 隔离性

undo 日志

将没有完成的事物回滚 (即撤销该事物的影响)

记录内容:

commit 表示事物成功完成

记录值的旧值 便于回滚

<u><T_i, start>:</u>	<u>transaction has begun</u>
<u><T_i, commit>:</u>	<u>completed successfully</u>
<u><T_i, abort>:</u>	<u>could not complete successfully</u>
<u><T_i, X, v>:</u>	<u>T_i has updated element X, whose old value was v</u>

记录过程:

事物都是先修改内存中的数据再输出到磁盘, 日志先记录在内存后输出到磁盘

1 事物对内存中数据修改前, 先记录相应操作

即先记录<t1,a,8>再修改 a 为 16

2 当内存中的修改要输出到磁盘前, 先将对应日志记录到磁盘上

即先将<t1,a,8>写到磁盘上的日志文件中, 再 output 修改磁盘中的 a

这里需要注意, 如果磁盘上记录了对应日志, 该磁盘上对应元素并不一定修改了, 例如磁盘上有<t1,a,8>, 磁盘上 a 的值可能还是 8, 因为日志写到磁盘上和修改磁盘上数据, 这两个操作不是连在一起执行的。

即未 commit 的事物, 即使操作记录到磁盘上, 元素不一定修改

但是如果磁盘上有 **commit** 记录，则对应元素一定修改了

eg:

Consider a system that uses pure undo logging. After a system crash, we find the following log entries on disk:

```
<START T1>; <T1,A,5>; <START T2>;  
<T1,B,1>; <COMMIT T1>; <T2,B,11>;  
<T2,C,8>; <COMMIT T2>; <START T3>;  
<T3,A,10>; <START T4>; <T4,A,11>;  
<T3,C,7>; <T4,B,22>
```

The last four actions in the schedule were incrementing each object by 1 (e.g. <T4,A,11> was updating A to 12).

What are the all of the possible values on disk for each of the database elements **A**?

- ☐ A 5, 10, 11, 12
- ☐ B 10, 11, 12
- ☐ C 11, 12
- ☐ D 12

t1 t2 提交 则 a 不可能为 5

a 可以是 10 11 12

3 当磁盘中数据修改完成后，才再磁盘日志中记录 **commit**

当 AB 在磁盘中修改完毕后，磁盘日志中记录 **commit**

Log is first written in memory
Not written to disk on every action

```
T1: Read (A,t); t ← t×2  
Write (A,t);  
Read (B,t); t ← t×2  
Write (B,t);  
Output (A);  
Output (B);
```

memory

```
A: 8  
B: 8  
Log:  
<T1,start>  
<T1, A, 8>  
<T1, B, 8>
```

DB

```
A: 8  
B: 8
```

Log



Database System - Nankai

日志规则:

- 1 所有修改操作都要记录旧值
- 2 修改磁盘元素前要把对应日志写到磁盘上
- 3 当所有 **output** 操作结束后，把 **commit** 写到磁盘日志上

- (1) Generate an undo log record for every update action (with the old value)
- (2) Before modifying element X on disk, any log records for X must be flushed to disk (write-ahead logging)
- (3) All WRITES of transaction T must be OUTPUT to disk before $\langle T, \text{commit} \rangle$ is flushed to log

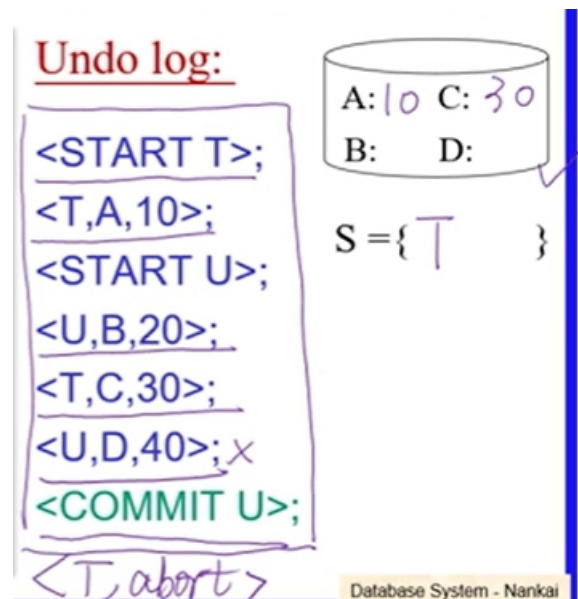
日志恢复:

如果日志中对应事物已经有 commit 或者 abort 则不同恢复
 对于没有结束符的日志，逆序进行操作将值恢复输出到磁盘，把对应事物的 abort 写到日志

- For every T_i with $\langle T_i, \text{start} \rangle$ in log:
 - If $\langle T_i, \text{commit} \rangle$ or $\langle T_i, \text{abort} \rangle$ in log, do nothing.
 - Else {
 - For all $\langle T_i, X, v \rangle$ in log:
 - write (X, v)
 - output (X)
 - Write $\langle T_i, \text{abort} \rangle$ to log

- (1) Let $S =$ set of transactions with $\langle T_i, \text{start} \rangle$ in log, but no $\langle T_i, \text{commit} \rangle$ (or $\langle T_i, \text{abort} \rangle$) record in log
 $S = (T_i, \dots)$
- (2) For each $\langle T_i, X, v \rangle$ in log, in reverse order (latest $\rightarrow$ earliest) do:
 - if $T_i \in S$ then {
 - write (X, v)
 - output (X)
- (3) For each $T_i \in S$ do
 - write $\langle T_i, \text{abort} \rangle$ to log

eg:



U 已经 commit 不回滚 逆序遍历 遇到 T 的操作将旧值写到磁盘
遍历到 T 的开始时，代表回滚结束，将对应的 abort 写到日志中

redo 日志：

和 undo 区别：

1 redo 记录修改的新值，回滚恢复的也是完成的事物

undo 记录旧值，回滚未完成的事物

2 redo 在遇到写盘时就要把日志包括 commit 全部写到磁盘，而 undo 是写一个元素，将对应记录写到磁盘上，写盘结束后才把 commit 写到磁盘上

记录过程：

1 事物对元素进行修改时，不用考虑日志和修改的先后顺序，只是要记录新值

2 当修改要写到磁盘前，现在内存日志中写上 commit 并写到磁盘中，再对磁盘值进行修改

记录规则:

- 1 修改都要记录, 记录更改后的值
- 2 事物要写盘前, 将该事物的日志包括 commit 写到磁盘上

日志恢复:

顺序扫描已经 commit 的日志, 再次执行操作

- For every T_i with $\langle T_i, \text{commit} \rangle$ in log:
 - For all $\langle T_i, X, v \rangle$ in log:
 - $\text{Write}(X, v)$
 - $\text{Output}(X)$

This time, the last updates should remain in effect

-
- (1) Let $S = \text{set of transactions}$
with $\langle T_i, \text{commit} \rangle$ in log $S = \{T_1, \dots, T_j\}$
- (2) For each $\langle T_i, X, v \rangle$ in log, in
forward order (earliest $\rightarrow$ latest)
do:
– if $T_i \in S$ then $\begin{cases} \text{Write}(X, v) \\ \text{Output}(X) \end{cases}$

注意点:

没有 commit 的事物, 对应的修改不可能输出到磁盘

commit 的事物, 修改不一定输出到磁盘

ckpt 之前的事物都已经写到磁盘 不需要恢复

检查点：

redo 日志恢复很慢(需要将做完的事物都做一遍)，因此设置检查点，检查点前的事物都为完成的，不需要再次执行

Redo log:



设置检查点的步骤：

- 1 停止接收新事物
- 2 等待正在执行的事物结束
- 3 将日志写到磁盘
- 4 将缓存区的更改信息写到磁盘
- 5 在内存和磁盘上写出检查点
- 6 进行事物过程

因此插入检查点的位置所有事物都已经完成：

undo/redo 日志：

记录内容：

记录元素的旧和新值

记录规则：

- 1 把数据写到磁盘前 先把日志记录写到磁盘上

2 数据写到磁盘可以在 commit 的之前或者之后（结合 undo redo）

记录过程：

- 1 对内存元素修改，每修改一个，都要在内存日志中记录
- 2 将数据写到磁盘前 先将对应记录写到磁盘日志中，再进行修改
- 3 事物修改完成后，commit 写到内存日志中，写到磁盘的时间随意

日志恢复：

有 commit 的事物 从前向后做一遍 将新值放入磁盘中

没有 commit 的事物 从后向前回滚 将旧值放入磁盘中

注意点：

由于 commit 和写到磁盘的顺序不确定，崩溃后元素值会有多种情况：

A 可能为 4 或者 5 ； B 为 9 或者 10

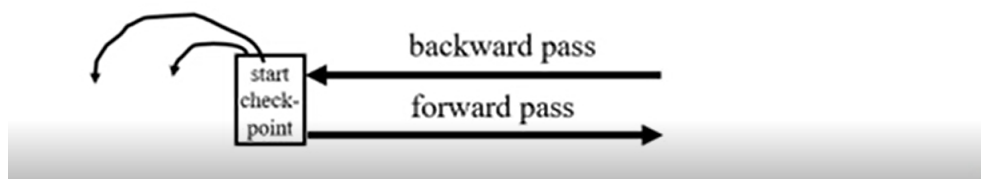
非静止的检查点：

插入时记录还未完成的事物，可能存在内存中的修改但是还未输出到磁盘上。
在记录中添加检查点结束标志

如果有 end check 则 start check 前的操作已经完成，即便没有 commit

流程:

- **Backwards pass** (end of log $\rightarrow$ latest checkpoint start)
 - construct set S of committed transactions
 - undo actions of transactions *not* in S
- **Undo pending transactions**
 - follow undo chains for transactions in (checkpoint active list) - S
- **Forward pass** (latest checkpoint start $\rightarrow$ end of log)
 - redo actions of transactions in set S



1 从日志结束逆序扫描磁盘上日志，找到最后一个检查点开始位置:

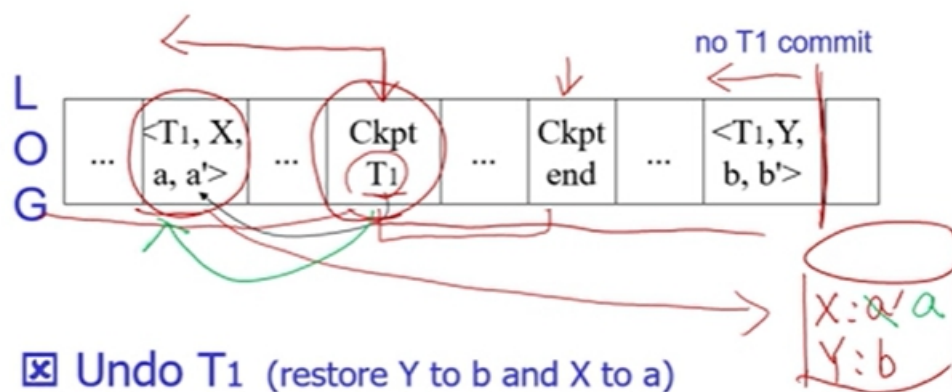
记录已经 commit 的事物，同时 undo 回滚未 commit 的事物

2 回滚到最后一个检查点开始位置时，将该位置活跃且最后未 commit 的事物继续回滚到事物开始时。

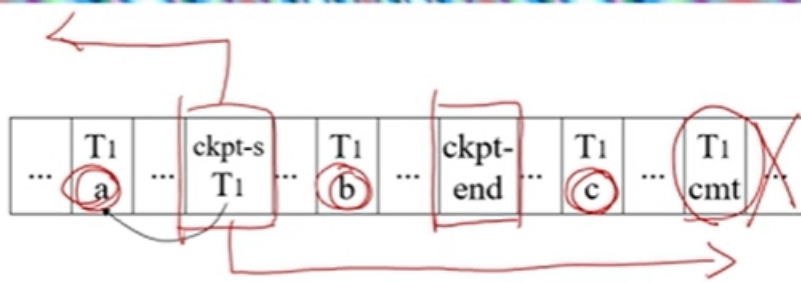
3 从最后检查点开始位置向后扫描，将 commit 的事物操作 redo 一遍

eg:

1



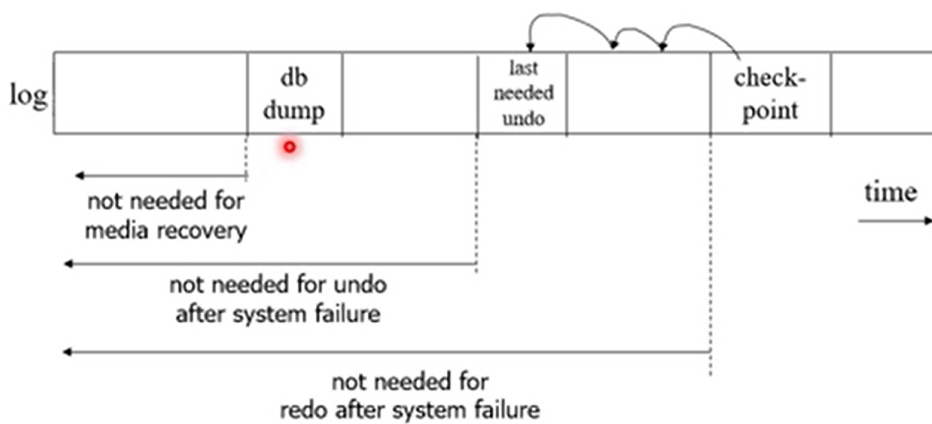
对于未 commit 的事物，回滚时，需要回滚检查点开始前的操作



☒ Redo T1: (redo update of elements to b and to c)

对应 commit 的事物，需要从检查点开始位置之后开始 redo

日志的备份：



第十章并发控制

(时间戳 有效性认证不考)

事物是读写操作组成的序列，不同事物同时对一个元素进行读写或者写写操作会造成冲突。

调度：

执行不同事物操作的顺序，针对不同元素的事物执行顺序不同会造成数据不一致
例如对 A 是 $t_1 t_2$ 而对 B 是 $t_2 t_1$ ，这样会导致数据不一致。

调度方式：

串行调度：不并发 操作不交叉

以下哪一个调度是串行调度（Serial Schedule）？

- ☐ A $T_1 = R_1(C) R_1(A) W_1(C) R_1(B) W_1(D) W_1(B) R_1(A)$
- ☐ B $S_1 = W_3(A) R_1(B) R_2(A) R_4(D) W_1(B) R_4(A) W_2(B) W_1(D) R_4(B)$
- ☐ C $T_2 = R_1(C) W_1(D) W_1(B) \quad T_{23} = R_2(A) W_2(D) W_2(B)$
- ☒ D $S_2 = W_3(A) R_1(B) W_1(D) W_1(B) R_2(A) W_2(B) R_4(D) R_4(A) R_4(B)$

S 为调度，串行调度不同事物不交叉，B 中为 312414214 交叉了 D 中 3124 无交叉

等价调度：两个调度的结果在任何数据库状态都相同，则二者等价
可串行化调度：等价于串行调度的调度

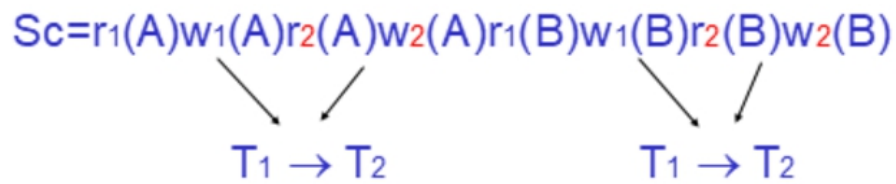
如果事物保持数据一致性，那么每个串行调度也保持一致性

冲突：

交换两个动作会改变调度语义 那么两个动作冲突

- 1 同一个事物的任何动作都不可以交换
- 2 不同的事物对同一个元素操作，其中一个动作是写操作，那么这两个动作冲突

引起冲突的事物要先后进行，可以构成事物的优先图



No cycles $\Rightarrow$ Sc is "equivalent" to a serial schedule
(in this case T_1, T_2)

冲突等价调度：

s1 可以被不冲突的转换成 s2（即转换不冲突的操作顺序）

可串行化调度：

操作可以不冲突的进行

重要结论：

结论 1：如果优先图没有环则为调度冲突可串行化，否则不是

结论 2：冲突等价调度的优先图一样，但是优先图相同的 not 一定是等价调度（没有环则成立）

求等价的串行化调度：

构造优先图，然后使用拓扑排序

两阶段锁：

规则 1： well-form 针对事物

访问数据前加锁 访问完解锁

规则 2： 合法调度 针对调度

对于一个元素只能加一个锁，以此控制访问顺序

规则 3： 两段锁协议

同一个事物加锁命令前不能有解锁 解锁命令后不能有加锁
因此一个事物 先把所有需要的锁加上，再逐个解锁

注意点：

仅仅满足规则 1 2 无法达到可串行化调度要求

满足规则 1 2 3 则为冲突可串行化调度

两段锁协议无法解决死锁问题

解决死锁的方法：

• 一次封锁法

- 要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行
- 存在的问题：降低系统并发度、难于事先精确确定封锁对象

• 顺序封锁法

- 顺序封锁法是预先对数据对象规定一个封锁顺序，所有事务都按这个顺序实行封锁。
- 顺序封锁法存在的问题：维护成本高、难以实现

共享/排他锁：

在读之前上共享锁 之后解锁 一个元素可以上多个共享锁

写之前上排它锁，之后解锁，一个元素上只能有一个排它锁（共享锁也不能有）

新的规则：

规则 1：

读之前上共享锁 写之前上排它锁，排他锁也可以用于读

针对于先读后写：

1 直接上写锁 缺点：降低并发度

2 先上读锁 遇到写操作更新为写锁

缺点：可能会延迟事物执行时间 因为升级为排他锁需要其他事物的解锁
同时有可能导致死锁

规则 2：

一个元素可以有多个读锁 但是只能有 1 个写锁

规则 3：两段锁协议

只有在加锁的阶段才可以升级锁 解锁的时候不可以加锁
其余和之前一样

更新锁：

更新共享锁为排他锁可能会导致死锁

解决方案：

某个元素被若干事物操作，第一个事物无法申请写锁 但是可以申请更新锁

以下是事物加锁可申请表：

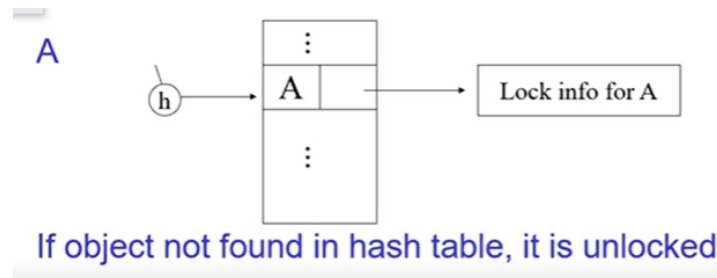
		New request		
Comp		S	X	U
Lock already held in	S	T	F	T ✓
	X	F	F	F ✗
	U	F ✗	F ✗	F ✗

注意事物有更新锁则无法再申请更新锁

已知先读后写，则直接申请更新锁

T1	T2	T1	T2
I-u ₁ (A);r ₁ (A)		I-S ₁ (A)	
I-X ₁ (A);w ₁ (A);u ₁ (A)	I-u ₂ (A)被拒绝	I-u ₁ (A);	I-S ₂ (A)
	I-u ₂ (A);r ₂ (A);		I-u ₂ (A)
	I-X ₂ (A);w ₂ (A);u ₂ (A)		

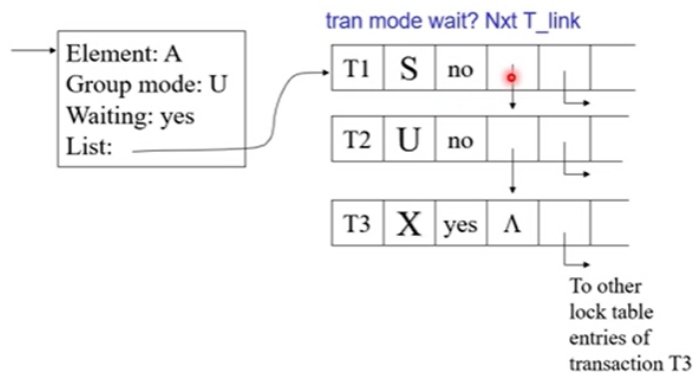
锁表的信息:



具体信息:

group mode 存放当前元素具有最高级别的锁

第一个指针指向下一个申请该元素锁的事物，第二个指针指向该事物持有或者正在申请的其他元素的锁



警示协议：

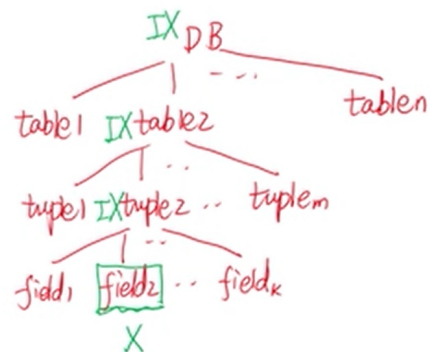
意向锁 intention lock:

修改元组中某个域

Hierarchically nesting elements (e.g. relation/block/tuple) can be locked with **intention locks** IS and IX

Idea

- start locking at the root (relation) level
- to place an S or X lock on a subelement, first place a corresponding intention lock IS or IX the element itself
 - Warns others: "I'll be reading/writing some subelement of this element"



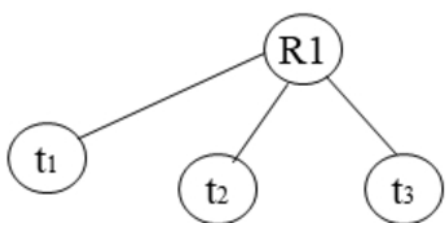
从根部开始加锁 先上意向锁 到修改节点后上对应的读或者写锁

注意兼容性

		IS	IX	S	X
Holder	IS	T	T	T	F
	IX	T	T	F	F
	S	T	F	T	F
	X	F	F	F	F

注意 IX 表示对表中某个 tuple 进行修改，如果再申请 IX，两者操作的都是 tuple 水平的，就是读写兼容问题，因此可以上 IX，其余的同理

如果是增加删减节点：

- Get an exclusive lock on A before deleting A
 - Insertions more problematic:
 - possible to lock only existing elements
 - Phantom tuples:
 - tuples that should have been locked, but did not exist when the locks were taken
- Use multiple granularity tree
 - Before insert of node Q, lock parent(Q) in X mode
- 
- ```
graph TD; R1((R1)) --- t1((t1)); R1 --- t2((t2)); R1 --- t3((t3))
```

插入的 tuple 不在表中，被称为幻影元组 phantom tuple，这样无法对该节点上锁

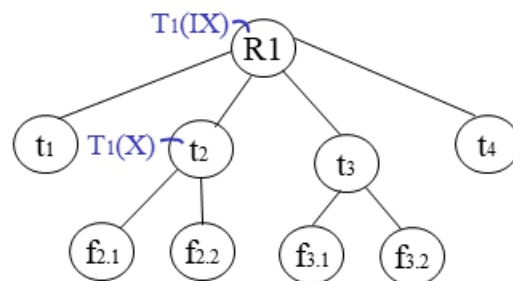
解决方案：对父节点上写锁

如果数据库元素的层次关系和持有锁的情况如右图所示， $T_2$ 事务插入 $T_5$ 元组，能否成功？

☐ A YES ☐ B NO

$T_2$ 事务应该对R1申请什么锁？

☐ C R1的 IX 锁  
☐ D R1的 X 锁



R1 上写锁

时间戳（不考）：

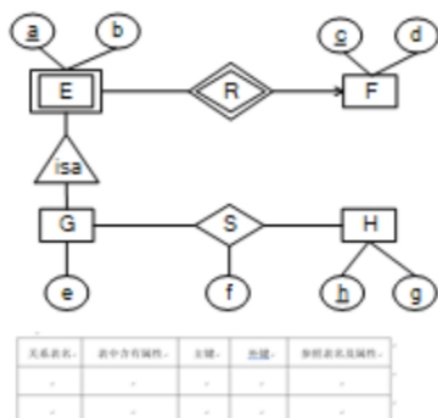
有效性验证（不考）：

# 考题样例：

## 题目 1：ER

1 ER 图转换为关系模型 2 写 sql 语句创建表 3 插入和删除（检查参照表）

应该掌握ER图中实体、属性、联系、弱实体集和子类的概念，可以将ER图转为满足3NF的关系模式，并深刻理解主键和外键的约束关系



Transform the E-R diagram into relation model (in 3NF)

Write SQL statements that create the tables (primary and foreign keys)

referential integrity :

On insert(S) -> exists(H) and exist(G);

On delete(F) -> delete(E) or not allowed

强实体中一对多 弱实体中有外键

注意表的合并 关系表的主键是相连实体表的主键

参照表是参照最邻近的那个表

关系表名 表中属性 主键 外键 参照表及属性

|   |         |     |       |                |
|---|---------|-----|-------|----------------|
| E | a b c   | a c | c     | F(c)           |
| F | c d     | c   |       |                |
| G | a c e   | a c | a c   | E(a) E(c)      |
| H | h g     | h   |       |                |
| S | a f c h |     | a c h | G(a) G(c) H(h) |

## 题目 2: sql

- 熟练使用关系代数和SQL语言操作数据库

- 根据SQL语句写出运行结果 Please write each result of the following queries
- 使用关系代数基本运算实现用户查询（掌握9种运算符即可）
- 能够使用SQL语句实现用户查询

通过 sql 语句写出对应的查询结果（注意别名）  
用关系代数和 sql 完成对应查询语句

## 题目 3: 范式

- 重点掌握3NF、BCNF，掌握保持无损连接和函数依赖的模式分解的算法

- FD and attribute closure
- keys and superkeys
- normal form(3NF,BCNF,4NF)

注意逻辑蕴含（函数依赖判断）

- 难点：函数依赖的分解，一定要在函数依赖的闭包集合上进行分解

例题：  $R(A, B, C)$ ,  $F=\{A \rightarrow B, B \rightarrow C\}$ .

$F$ 的闭包中含有 $A \rightarrow C$ ,  $F$ 逻辑蕴含 $A \rightarrow C$ .

分解两个关系模式：  $R_1(A,B)$ ,  $R_2(A,C)$ ,  $R_2$ 上存在函数依赖  $A \rightarrow C$ , 千万不能丢

作业：设 $R(A, B, C, D)$ ,  $F=\{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$ ,  $R$ 的候选码有：\_\_\_\_\_。对分解  $r = \{R_1(A, B), R_2(B, C), R_3(C, D)\}$ , 它\_\_\_\_\_（是/不是）无损连接, \_\_\_\_\_（保持/不保持）函数依赖。

正确答案：  $B^+ = \{C, D, A, B\}$ ,  $F$ 逻辑蕴含 $B \rightarrow A$ , 同理 $F$ 逻辑蕴含 $C \rightarrow B, D \rightarrow C$

$R_1(A, B)$ ,  $F_1=\{A \rightarrow B, B \rightarrow A\}$ ;  $R_2(B,C)$ ,  $F_2=\{B \rightarrow C, C \rightarrow B\}$ ;  $R_3(C, D)$ ,  $F_3=\{C \rightarrow D, D \rightarrow C\}$

保持函数依赖, 保持无损连接

2nf 1nf 不考 只考 3nf 及以上

函数依赖： (商店编号, 商品编号) → 部门编号;  
(商店编号, 部门编号) → 负责人;  
(商店编号, 商品编号) → 库存数量

候选码： (商店编号, 商品编号)

主属性： 商店编号, 商品编号; 非主属性： 部门编号, 负责人, 库存数量

没达到3NF: 对于 (商店编号, 部门编号) → 负责人, 左边不是superkey, 右边不是主属性

达到2NF: (商店编号)<sup>+</sup> = { }; (商品编号)<sup>+</sup> = { }

商店编号和商品编号单独不能推出任何属性, 不存在非主属性对候选码的部分依赖

2nf 不存在非主属性对候选码的部分依赖

考题:

Consider a relation  $R = (\dots)$  with FD's  $\dots$

- What is the attribute **closure** of \*\*\*?
- Of the following FDs, circle the ones that are **implied by the functional dependencies** given above.
- List **all keys** for R.
- Write down two functional dependencies that causes this relation to **violate NF**.
- We decompose R into  $R_1(\dots)$  and  $R_2(\dots)$ . What are the keys of  $R_1$ ? What are the keys of  $R_2$ ?
- Decompose R into two or more relations that are all in 3NF. And make sure your decomposition is (i) dependency preserving, and (ii) lossless join.

属性闭包 (求码 判断函数依赖)

求出码

违背范式的函数依赖 违背不同范式导致的后果 (插入异常 更新异常 删除异常)

分成 3nf 的表 (注意不同关系表不能包含)

判断分解后的 key 函数依赖的拆分方式

## 题目 4: 数据故障恢复

- 掌握日志的记录方式: What are the all of the possible values on disk for each of the database elements A, B and C?
- 掌握日志的恢复过程: Which, if any, transactions will need to be redone and undone in the recovery process?
- 知道日志恢复后数据库的正确状态: If finished the system recovery, what are the values on disk for each of the database elements A, B and C

Consider a system that uses undo-redo logging. After a system crash, we find the following log entries on disk:

|                  |              |
|------------------|--------------|
| <START T1>       | <COMMIT T2>  |
| <T1,A,5,0>       | <END CKPT>   |
| <START T2>       | <START T3>   |
| <T1,B,1,2>       | <T3,A,0,10>  |
| <COMMIT T1>      | <START T4>   |
| <T2,B,2,3>       | <T4,A,10,11> |
| <START CKPT(T2)> | <T3,C,9,7>   |
| <T2,C,8,9>       | <T4,B,3,22>  |

What are the all of the possible values on disk for each of the database elements **A**?

- ☐ A 5,0,10,11
- ☐ B 0,10,11
- ☐ C 10,11
- ☐ D 11

内存中记录未写到磁盘

恢复后的值 需要 redo 和 undo 的值

## 题目 5：并发控制

- 若干事务并发运行，能够判断所给的调度是不是可串行化调度（优先图）
- 如果已知几个事务，如何添加合适的锁，保证是可串行化调度（两段锁协议 多粒度锁）
- 了解死锁是怎样形成的



## 作业练习

For each of the following schedules, answer the questions below:

$S_b = W_3(A) R_1(B) R_2(A) R_4(D) W_1(B) R_4(A) W_2(B) R_1(D) R_4(B)$

- (a) What is the precedence graph for the schedule  $S_b$ ?
- (b) Is the schedule conflict serializable? If so, show all equivalent serial transaction orders. If not, describe why not.

给定调度 写出优先图 判断是否为冲突可串行化调度  
读写冲突需要先后顺序 读和读之间不用判定顺序

冲突可串行化调度（拓扑排序 有多个结果）

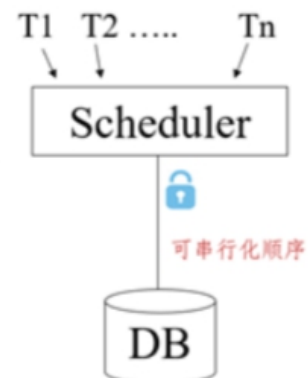
Consider the following two transactions:

$T_1 = R_1(C) R_1(A) R_1(B) W_1(A) W_1(C);$

$T_2 = R_2(B) W_2(A) R_2(A) R_2(C) W_2(C);$

(a) 请添加合适的加锁和解锁命令，使事务 $T_1$ 和 $T_2$ 在并发运行时可以保证数据库的一致性；

(b) 请说明这两个事务会引起死锁吗？如果会引起死锁，请给出死锁的示例；如果不会引起死锁，请说明为什么？



## 题目 6：求函数依赖和 key

Consider a relation  $R = (A, B, C, D, E)$  with

FD' s  $A \rightarrow C$ ,  $CD \rightarrow B$ ,  $B \rightarrow E$  and  $E \rightarrow D$

(a) What is the attribute closure of  $CD$ ?

(b) Of the following FDs, circle the ones that are implied by the functional dependencies given above:

i.  $AB \rightarrow D$     ii.  $CD \rightarrow A$     iii.  $B \rightarrow D$     iv.  $AD \rightarrow C$

(c) List all keys for  $R$ .

(a)  $(CD)^+ = \{C, D, B, E\}$

(b)  $\checkmark$  i.  $AB \rightarrow D$     ii.  $CD \rightarrow A$

$\checkmark$  iii.  $B \rightarrow D$

$\checkmark$  iv.  $AD \rightarrow C$

(c)  $AB, AD, AE$